Bsc Final Year Project Retrospective
Full unit Project
A concurrency Based Game Environment

Henry Frodsham - 101050430
Royal Holloway University of London
Department of Computer Science
Supervisor: Felix Bradley

December 2025

Contents

1 Comparison to initial Project plan

The development of this project throughout term 1 has differed significantly compared to the original project plan.

1.1 Major Difference 1: Event Architecture Implementation Schedule

The timeline for implementing event architecture outlined in my project plan is as follows:

Week 2: Research efficient event architecture.

Week 4: Code central event bus.

Week 6: Implement event queue.

However, the actual order of implementation during the development of the project was:

Week 1: Research efficient event architecture.

Week 2: Implement event bus and event queue architecture.

The substantial difference between the planned and actual implementation timeline is due to underestimating how crucial event architecture would be to my project. When initially creating the project plan, I had not anticipated the role event architecture would play in facilitating class communication within the project. For example, regarding input handling, I initially expected each game instance to both read and process game input independently. However, in the currently implemented system, I have a single class that reads input and dispatches raw input events to each game instance through am event queue. This architectural decision necessitated earlier implementation of the event system than originally planned.

1.2 Major Difference 2: ECS Registry Class Implementation

The anticipated timeline for implementing an ECS registry class is as follows:

Week 4: Code ECS registry class.

Week 7: Code entity factories and create entity components for ECS.

However, the actual implementation order for creating the ECS registry class was:

Week 11: Create an ECS registry class with demo implementation.

Week 11: Create ECS components with an ECS factory.

Although the timeline gap appears substantial, it does not represent a significant deviation from my intended development progression. I had planned for ECS to be implemented towards the end of the development cycle in order to produce a demonstration of the technology with a sample game object. Therefore, the timeline gap is primarily attributable to underestimating the architectural groundwork required before ECS implementation could begin. Specifically, the input handling system and display overlay needed to be fully implemented

and tested before the ECS architecture could be properly integrated into the project. This foundational work took longer than initially anticipated, pushing the ECS implementation to Week 11 rather than the originally planned Week 4-7 time frame.

1.3 Major Difference 3: Split Screen Implementation

The anticipated timeline for implementing split screen within my project is:

Week 6: Code initial split screen functionality.

However, the actual timeline throughout Term 1 was:

Week 12: Initial split screen functionality.

A significant factor contributing to this six-week gap between the anticipated and actual timeline is the delays in implementing prerequisite functionality within my project. Implementing split screen without visible objects on screen would have been impractical; therefore, not only did the ECS system need to be implemented, but also a streamlined method for creating ECS entities and rendering those objects on screen.

Another reason for the delayed split screen functionality was the unexpected amount of time required to implement the overlay system. I had anticipated this to be a relatively straightforward task due to the ease of use typically associated with Dear ImGui. However, this assumption proved incorrect, and the implementation consumed considerably more development time than originally allocated. These delays resulted in split screen functionality being postponed until Week 12, six weeks behind the initial schedule.

1.4 Major Difference 4: The Absence of Game Functionality

The anticipated timeline for implementing a partially functional game is as follows:

Week 8-9: Create partial game functionality.

However, creating partial game functionality throughout Term 1 proved unfeasible due to delays in other critical components of the project, namely the overlay system and input handling architecture. These foundational systems needed to be fully operational before game logic could be meaningfully implemented.

Despite this setback, the delay in implementing game functionality may ultimately prove beneficial to the project. During Term 1, I identified the Resource Entity Action (REA) pattern as a potentially valuable architectural approach for managing game state and logic. Consequently, implementing game functionality in Term 2 will allow for cleaner integration with this pattern, rather than requiring significant refactoring of hastily implemented game logic. This approach should result in a more maintainable and well-structured codebase.

2 Reflection

2.1 What went well

Development of this project throughout Term 1 had multiple achievements that went beyond my initially set goals. These achievements involve either more robust implementations or additional functionality not considered during initial planning. I am satisfied with the current overall state of the program, as many large architectural tasks have been implemented in preparation for Term 2, meaning that implementing RTS functionality will require less groundwork.

Input handling was a significant success throughout the development in Term 1. The currently implemented input handling allows users to unplug their controller whilst the project is running and then reconnect at any time to resume control of their allocated viewport. In addition, the program can automatically recognise newly plugged in devices and configure them accordingly without requiring a restart or manual intervention.

Another success throughout Term 1 was the split screen functionality. The program can automatically create new viewports when new devices are connected and resize existing viewports as they are added or removed. The currently implemented split screen functionality also maintains a consistent aspect ratio for the displayed game world without scaling issues. This was particularly satisfying as I had initially expected to encounter stretching of game objects depending on the size and shape of each viewport, but the implementation successfully avoided these problems.

2.2 What went wrong and why

During development of this project in Term 1, there were some interruptions that caused significant delays in following the development timeline. These interruptions did not halt development entirely and alternative solutions were found.

One major interruption in Term 1 was implementing an overlay system. The initial plan was to use Dear ImGui as an overlay system for this project, however development proved that Dear ImGui is extremely difficult to integrate into Ogre3D. The main reason for this oversight is that at a surface level, Ogre3D appears to support Dear ImGui natively and includes examples on their repository. The major issue encountered was that the given example assumes use of Ogre3D's ApplicationContextBase system, which is a means of simplifying Ogre3D applications. The underlying problem was that implementing ApplicationContextBase would undermine this project's control over Ogre3D. This would not only trigger a refactor of pre-existing code but would also affect my ability to fine tune Ogre3D's rendering to better fit the concurrency focus of the

project. As a result, an alternative overlay solution had to be researched and implemented, causing delays to the original timeline.

Another major interruption in Term 1 was the result of a limitation in SDL2. The controllers used to test the input handling in my project were generic off-brand controllers without a unique vendor ID or product ID. The absence of a vendor or product ID in the controllers used led to an unforeseen issue where distinguishing between each controller after they have been disconnected is impossible. This issue does not impact the behaviour of the program whilst the controllers are already connected, however two controllers disconnecting at the exact same time can lead to a misallocation once both are reconnected. This limitation required additional testing with different controller types and consideration of edge cases that were not originally accounted for in the development timeline.

2.3 Considerations for future development

After reflecting on the deficiencies of my previous plan, I have realized that more research and analysis of open source programs implementing similar technologies is required to prevent delays attributed to unforeseen issues. If I had analysed other implementations of Dear ImGui with Ogre3D before starting development, then the disruption could have been avoided entirely or dealt with in a more timely manner. This demonstrates the importance of thoroughly investigating existing solutions and their integration challenges before committing to specific technologies in the development plan.

A major lesson learned from Term 1's development is that more careful testing is required across various hardware configurations. SDL2's limitation where generic controllers can't be distinguished was only realised due to my use of generic controllers. If I had been using branded controllers with unique vendor and product IDs, this limitation would have gone unnoticed and would have had major implications for end users in the future. Moving forward, I will ensure testing is conducted with a wider range of hardware to identify similar edge cases earlier in development.

Another aspect I will consider during Term 2's development is a more spaced out workload for writing final reports. Throughout Term 1, the writing of reports was inconsistent and only the project diary and control flow diagram were written as planned. To ensure this does not happen again, I will write sections of the report alongside the diary entries, making sure to set sections of the report as branch objectives before merging into main. This approach will integrate documentation into the regular development workflow rather than treating it as a separate task that can be postponed.

3 Term 2 Timeline

following my reflection of how the initial project timeline translated into actual implementations throughout term 2, i have produced a timeline for term 2

Week 1: revisit term 1 implementation, refactor threading model and class communication
Weeks 1-2: Research REA implementations, implement a basic scenario with multiple entities that can perform basic actions.
Weeks 3-4: create a game world and a basic test scenario where each player is given starting units and starting territory
Weeks 4-5: model and create simple REA actions for taking territory, attacking units, placing units.
Weeks 6-7: create an undecorated UI, including context menus for actions that change depending on where the cursor is. settings and main menu screen
Weeks 7-8: perform market analysis on experienced real time strategy players and implement suggestions as prototypes
Weeks 8-9: perform further surveys of prototype combinations of these suggestions to produce the unrefined playable game
Weeks 9-12: produce a shader that complements the simplistic mesh's, produce a simplistic main menu screen and UI design to produce the finalised product.
(Time points overlap with each other to account for unexpected overhead)

4 Risk assessment and mitigation

4.1 Risk 1: imbalance between keyboard and controller

Real time strategy games typically arent implemented with controller support for the reason of an unfair advantage between a player on keyboard and a player on controller. typically, the keyboard player can react faster and thus gains an edge in the RTS genre. to combat the imbalance between these 2 input devices I will design the attacking and expansion mechanics around ease of use for both input devices, the main differences i will implement differing to common RTS games is quick actions accessible through the controller D-pad which will update depending on the cursor position, e.g if the controller player is selecting inside there own territory then the quick actions could be "build x building here" or if they select an area close to a foe then "attack here".

4.2 Risk 2: inadequate use of concurrency

Throughout development in Term 2, its possible that there's a major imbalance in processing between the game instance threads and the main application thread. whilst a slight imbalance is inevitable due to ogre3d running on the main thread there should still be a clear spread of processing load over the game instance threads. to ensure adequate spreading of workload i will monitor cpu core utilization whilst running prototype implementations of my project and adjust my implementation of concurrency to produce a relatively even spread of CPU workload across each core.

4.3 Risk 3: homogenous testing environment

This risk will become more important to avoid through this projects development in term 2. its imperative that the performance of my project doesn't differ more than expected when running across different platforms or hardware combinations. to mitigate this risk, I will test this project across 2 different computers with one running Windows 11 and one running Linux, the hardware of each computer can be seen below:

Primary Testing and Development environment
Operating System: Windows 11 Home edition
CPU: Ryzen 9 9950X3D
GPU: Nvidia RTX 5090
RAM: 64GB DDR4
disk running and storing project: Nvme SSD

secondary testing environment
Operating System: Linux Mint with Wine installed
CPU: Ryzen 7 3700x
GPU: AMD Radeon 7700XT
RAM: 16GB DDR4

disk running and storing project: standard sata SSD

(both running at factory clock rates)

4.4 Risk 4: inability to implement REA

This risk is especially important for Term 2's development due to the previous failure to implement Dear Imgui. to prevent repeating the resulting delay to the development Timeline I will research REA implementations thoroughly and look for projects implementing both REA and ECS that are more relevant to my projects development and note which REA C++ library is standard for the entt library (for ECS). In the case of a previously unseen issue arising between ECS and REA compatibility I will research common workarounds or alternatives to implement instead, thus minimizing the final delay to the project timeline.

5 Acronomys

ECS - Entity Component System
REA - Resource Entity Action
RTS - Real Time Strategy
SDL2 - Software Direct Media Layer
LCM - Local Multi Player
CPU - Central Processing Unit
GPU - Graphical Processing Unit
RAM - Random Access Memory

6 Glossary

Ogre3d - a lightweight rendering engine in the form of a c++ library
Dear Imgui - a c++ rendering library focused on creating debug overlays
Software Direct Media Layer - a commonly used cross platform input handling library for c++
Resource Entity Action - a design pattern for real time strategy games that models entity interactions into transformation matrices that define resource exchanges between entities for fast evaluation
Entity Component System - a software architecture pattern that seperates game objects into entities (identifiers) and components (pure data containers). ECS is used where composition is prioritized over inheritance
entt - an Entity Component System library for C++ that uses sparse set to store entities

7 References